COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S1
DATE PERFORMED:	10/20/26	**DATE SUBMITTED:**	10/22/26
MEMBER/S:	Medina, Jomz Ryle E.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 14.1 **Algorithm Complexity**

1. PROCEDURE OUTPUT

Output/s:

Pseudocodes

Bubble sort:

```
/*
bubbleSort(arr)
    n = length of arr
    for i from 0 to n - 2
        swapped = false

        for j from 0 to n - i - 2
            if arr[j] > arr[j + 1] then
                swap arr[j] and arr[j + 1]
                swapped = true
            end if
        end for

        if not swapped then
            break
        end if
    end for
end
*/
```

Shell sort:

```
/* FOLLOWING LAB PSEUDOCODE:
shellSort(array, size)
    for interval i <= size/2n down to 1
        for each interval "i" in array
            sort all the elements at interval "i"
end shellsort
*/
```

	Best Case	Worst Case	Analysis
Bubble Sort	O(n)	O(n)^2	The way bubble sort works is that it repeatedly compares adjacent elements together and swaps them if the case is that they are in the wrong order. The best case happens when the array is already sorted which results in a time complexity of O(n). On the other hand, the worst case happens when the array is arranged in a reversed order, resulting in O(n^2). Thus, we can make the logical deduction that Bubble Sort becomes less efficient as the size of the input increases.
Shell Sort	O(n log n)	O(n)^2	Shell sort, compared to bubble sort, makes it so that elements are compared at different gaps instead of the immediate adjacent values. For the best case, time complexity goes down to O(n log n) with how the gap goes down until fully sorted. Furthermore, the worst case happens with O(n^2) conditionally between the sequence of the gap as well as how the elements are arranged. Thus, we can say that Shell Sort must be more efficient, being able to move elements closer to correct positions earlier than bubble sort.

Table 14-1. Best- and Worst-Case Analysis using Theoretical Tools

Code Screenshot:

```cpp
#include <iostream>
#include <vector>
#include <algorithm>
#include <chrono>
#include <iomanip>



void bubbleSort(std::vector<int>& arr) {
    int n = arr.size();
    bool swapped;

    for (int i = 0; i < n - 1; i++) {
        swapped = false;
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                std::swap(arr[j], arr[j + 1]);
                swapped = true;
            }
        }

        if (!swapped)
            break;
    }
}
```

```cpp
void shellSort(std::vector<int>& arr) {
    int n = arr.size();

    for (int gap = n / 2; gap > 0; gap /= 2) {
        for (int i = gap; i < n; i++) {
            int temp = arr[i];
            int j = i;

            while (j >= gap && arr[j - gap] > temp) {
                arr[j] = arr[j - gap];
                j -= gap;
            }

            arr[j] = temp;
        }
    }
}
```

```cpp
double measureTime(const std::vector<int>& original, bool useBubbleSort) {
    const int trials = 5;
    double totalTime = 0;

    for (int t = 0; t < trials; t++) {
        std::vector<int> arr = original;

        auto start = std::chrono::steady_clock::now();

        if (useBubbleSort) {
            bubbleSort(arr);
        } else {
            shellSort(arr);
        }

        auto finish = std::chrono::steady_clock::now();

        double microseconds =
            std::chrono::duration<double, std::micro>(finish - start).count();
        totalTime += microseconds;
    }

    return totalTime / trials;
}
```

```cpp
int main() {
    int sizes[] = {100, 1000, 10000};

    std::cout << std::fixed << std::setprecision(2);
    std::cout << "LAB 14.1 - EXPERIMENTAL SORTING ANALYSIS PROCEDURE\n";
    std::cout << "Average execution time in 5 trials (microseconds)\n";

    for (int n : sizes) {
        std::vector<int> bestCase;

        // Best case
        for (int i = 1; i <= n; i++) {
            bestCase.push_back(i);
        }

        // worst case
        std::vector<int> worstCase = bestCase;
        std::reverse(worstCase.begin(), worstCase.end());

        double bubbleBest = measureTime(bestCase, true);
        double bubbleWorst = measureTime(worstCase, true);
        double shellBest = measureTime(bestCase, false);
        double shellWorst = measureTime(worstCase, false);

        std::cout << "\nInput Size (N): " << n << "\n";
        std::cout << std::left << std::setw(16) << "Case"
             << std::setw(23) << "Bubble Sort (us)"
             << "Shell Sort (us)\n";
        std::cout << std::left << std::setw(16) << "Best Case"
             << std::setw(23) << bubbleBest << shellBest << "\n";
        std::cout << std::left << std::setw(16) << "Reverse Sorted"
             << std::setw(23) << bubbleWorst << shellWorst << "\n";
    }
    return 0;
}
```

Output:

```
LAB 14.1 - EXPERIMENTAL SORTING ANALYSIS
Average execution time of 5 trials (microseconds)

Input Size (N): 100
Case            Bubble Sort (us)        Shell Sort (us)
Best Case       0.64                    4.62
Reverse Sorted  85.24                   7.32

Input Size (N): 1000
Case            Bubble Sort (us)        Shell Sort (us)
Best Case       5.58                    70.70
Reverse Sorted  8778.20                 117.42

Input Size (N): 10000
Case            Bubble Sort (us)        Shell Sort (us)
Best Case       54.96                   1061.80
Reverse Sorted  854418.00               1767.44
```

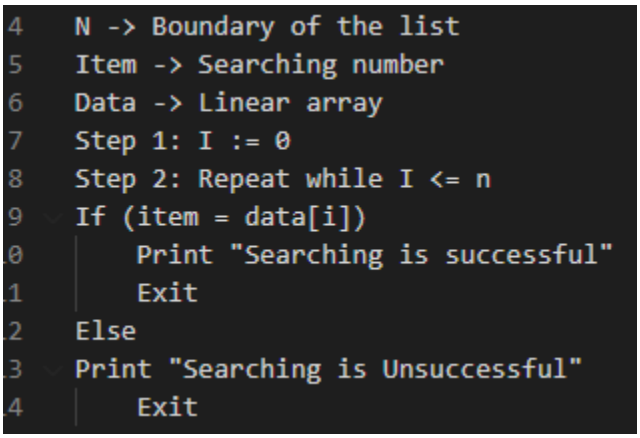
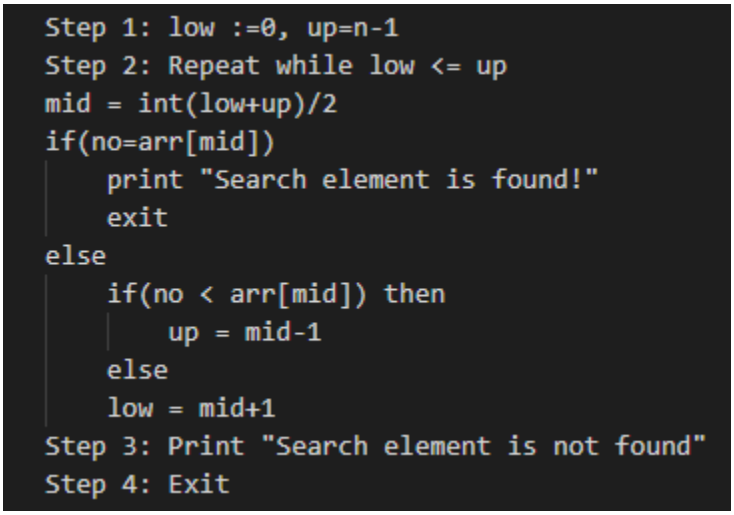
	Bubble Sort (µs)			**Shell Sort (µs)**		
Best Case	100	1,000	10,000	100	1,000	10,000
	0.64	5.58	54.96	4.62	70.70	1061.80
Worst Case	100	1,000	10,000	100	1,000	10,000
	85.24	8778.20	854418.00	7.32	117.42	1767.44

Table 14-2. Best- and Worst-Case Analysis using Experimental Tools

Analysis:

The theoretical and experimental results match exactly what was predicted where bubble sort seems to deal much worse with a higher input size **(n)** in comparison with shell sort which is very efficient despite the major jump from 1000 to 10000.

2. SUPPLEMENTARY OUTPUT

Pseudocodes	
Linear Search	
Binary Search	

```
N -> Boundary of the list
Item -> Searching number
Data -> Linear array
Step 1: I := 0
Step 2: Repeat while I <= n
If (item = data[i])
    Print "Searching is successful"
    Exit
Else
Print "Searching is Unsuccessful"
    Exit
```

```
Step 1: low :=0, up=n-1
Step 2: Repeat while low <= up
mid = int(low+up)/2
if(no=arr[mid])
    print "Search element is found!"
    exit
else
    if(no < arr[mid]) then
        up = mid-1
    else
    low = mid+1
Step 3: Print "Search element is not found"
Step 4: Exit
```

	Best Case	**Worst Case**	**Analysis**
Linear Search	O(1)	O(n)	Basically just examines all elements one by one until the specified target is found (best case if it's the first) or the end of the array is reached (worst case). By checking each individual element, linear search may be worse than Binary Search here, however, still has much functionality with unsorted data structures.

Binary Search	O(1)	O(log n)	The code used for this implementation of binary search includes a recursive structure where the middle element is repeatedly checked and about half of the remaining elements are let off. Binary search is much more efficient, but would need an already sorted array to properly work.

Code Screenshot:

```cpp
// CPE010 - Lab 14.1 Supplementary Activity

#include <iostream>
#include <vector>
#include <chrono>
#include <iomanip>

using namespace std;

// Linear Search from lab
bool linearSearchArr(const vector<int>& data, int item){
    for (int i = 0; i < (int)data.size(); i++){
        if (data[i] == item) {
            return true;
        }
    }
    return false;
}

// Binary Search from lab
bool binarySearchArr(const vector<int>& arr, int low, int high, int item){
    if (low > high) {
        return false;
    }

    int mid = low + (high - low) / 2;

    if (arr[mid] == item){
        return true;
    } else if (item < arr[mid]){
        return binarySearchArr(arr, low, mid - 1, item);
    } else {
        return binarySearchArr(arr, mid + 1, high, item);
    }
}
```

```cpp
double measureTime(const vector<int>& arr, int item, bool useBinary, int repetitions) {
    const int trials = 5;
    double total = 0;

    for (int trial = 0; trial < trials; trial++) {
        volatile int foundCount = 0;
        auto start = chrono::steady_clock::now();

        for (int i = 0; i < repetitions; i++) {
            bool found;

            if (useBinary) {
                found = binarySearchArr(arr, 0, (int)arr.size() - 1, item);
            } else {
                found = linearSearchArr(arr, item);
            }

            if (found) {
                foundCount++;
            }
        }

        auto finish = chrono::steady_clock::now();

        double microseconds =
            chrono::duration<double, micro>(finish - start).count();
        total += microseconds / repetitions;
    }

    return total / trials;
}
```

```cpp
int main() {
    int sizes[] = {1000, 10000, 100000};

    cout << fixed << setprecision(4);
    cout << "LAB 14.1 - SUPPLEMENTARY ACTIVITY\n";
    cout << "Average execution time of 5 trials (microseconds)\n";

    cout << left << setw(18) << "Input Size (N)" << setw(26) << "Binary Search (us)" << "Linear Search (us)\n";

    for (int n : sizes) {
        vector<int> data;
        for (int i = 1; i <= n; i++) {
            data.push_back(i);
        }

        int target = n + 1;
        int linearRepetitions = 5000000 / n;
        int binaryRepetitions = 100000;

        double binaryTime = measureTime(data, target, true, binaryRepetitions);
        double linearTime = measureTime(data, target, false, linearRepetitions);

        cout << left << setw(18) << n << setw(26) << binaryTime << linearTime << "\n";
    }

    return 0;
}
```

Output:

```
LAB 14.1 - SUPPLEMENTARY ACTIVITY
Average execution time of 5 trials (microseconds)
Input Size (N)    Binary Search (us)       Linear Search (us)
1000              0.0904                   4.8802
10000             0.1379                   49.4294
100000            0.1628                   494.1004
```

Input Size (N)	Elapsed Time for Binary Search (Worst-Case)	Elapsed Time for Linear Search (Worst-Case)
1,000	0.0904	4.8802
10,000	0.1379	49.4294
100,000	0.1628	494.1004

Experimental Analysis:

Based on the results of the experimental analysis, linear search seems to grow much faster compared to binary search, directly matching the results for the theoretical analysis. At n = 100,000, linear goes for 494.1004 microseconds while binary search only went for 0.1628 microseconds. Therefore, binary search seems to grow much more efficiently than with linear search as it repeatedly cuts the search area in half while linear searches through each element.

Conclusion:

In this activity, I learned a lot about how algorithm complexity could affect the execution speed as well as efficiency of different algorithms. From comparing bubble sort to shell sort and linear to binary, I overall got the main message of how choosing the appropriate algorithm is important even if the two algorithms may produce the same exact result. Additionally, theoretical analysis not only helps with understanding the expected result, but also having a firmer understanding of the algorithms that you plan to implement.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **None**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

COMPUTER ENGINEERING

QUEZON CITY

Purpose of Use: **N/A**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **N/A**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.